FPT UNIVERSITY | FPTU QUY NHON AI CAMPUS FU CAHATA • REFERENCE DOCS

📺 Plan Xử Lý Video Streaming — OmniCast (PRN232 + PRM393)

Mục tiêu: Tổng hợp 12 góp ý từ Review 1 về video streaming thành một kế hoạch triển khai có thứ tự ưu tiên rõ ràng, phân chia task theo đúng phạm vi PRN232 (Backend + Web) và PRM393 (Mobile Flutter).
Phạm vi áp dụng: MVP cho Demo nhiều người xem cùng lúc (≥ 50 viewers), 5-7 kênh chạy tuần tự, gói Subscription mô phỏng, phân quyền minh bạch theo Role RBAC.
Cập nhật: Oct 2026 — dựa trên codebase hiện tại (OmniPlayer, VideoPlayer, StitchLivePlayer, ChannelTiers đã có sẵn).

🎯 1. Tổng Quan 12 Góp Ý & Trạng Thái Xử Lý

#	Góp Ý	Hiện trạng Code	Hành động Cần làm	Mức ưu tiên
1	AI phát video	❌ Chưa có — AI chỉ thẩm định nội dung	Tích hợp Auto-Curator AI chọn video + lập lịch phát tuần tự	🟢 Cao
2	Nguyên nhân Lag & luồng dữ liệu	⚠️ Mobile dùng `video_player` gốc, Web dùng `hls.js` — chưa có CDN/edge	Dựng kiến trúc CDN + Adaptive Bitrate	🟢 Cao
3	Ghi log	⚠️ Backend có `AuditLogger` (gRPC) cho CRUD, chưa log stream	Mở rộng audit + log view metric cho streaming	🟡 Trung bình
4	Kỹ thuật Streaming phù hợp	⚠️ HLS qua CDN, chưa có ABR đầy đủ	Chọn **HLS + ABR** làm chuẩn, bổ sung fallback MP4	🟢 Cao
5	Nguồn Video	⚠️ Đang dùng YouTube embed + Mux test stream	Chuyển sang **CDN self-hosted (Cloudflare/Bunny)** + ingest từ nguồn công khai	🟢 Cao
6	Học VTVGo & áp dụng	❌ Chưa nghiên cứu	Reverse-engineer flow VTVGo → apply	🟢 Cao
7	5-7 kênh tuần tự	⚠️ Đã có 25 channel tier (5 free, 20 VIP)	Roster gọn: 5-7 kênh, chạy sequential playlist	🟢 Cao
8	Live	⚠️ Có `LIVE NOW` UI, chưa pipeline thật	Pipeline RTMP ingest → HLS packager	🟢 Cao

#	Góp Ý	Hiện trạng Code	Hành động Cần làm	Mức ưu tiên
9	Share	⚠️ Có `ShareHelper` cơ bản	Mở rộng share kiểu VTVGo (Deep link + QR + Social)	🟡 Trung bình
10	Phân quyền rõ ràng	⚠️ Có RBAC 1/2/3, chưa map chi tiết ở tầng stream	Bổ sung `streamPermission` cho từng kênh + middleware kiểm tra	🟢 Cao
11	Pay Subscription	⚠️ Có VIP Demo Mock QR, chưa có payment thật	Mock payment gateway (Sandbox) + webhook	🟡 Trung bình
12	Demo nhiều người xem	❌ Chưa có kế hoạch	Kiến trúc scale: CDN edge + WebSocket realtime	🟢 Cao

🔬 3. Nghiên cứu VTVGo — Rút ra bài học áp dụng

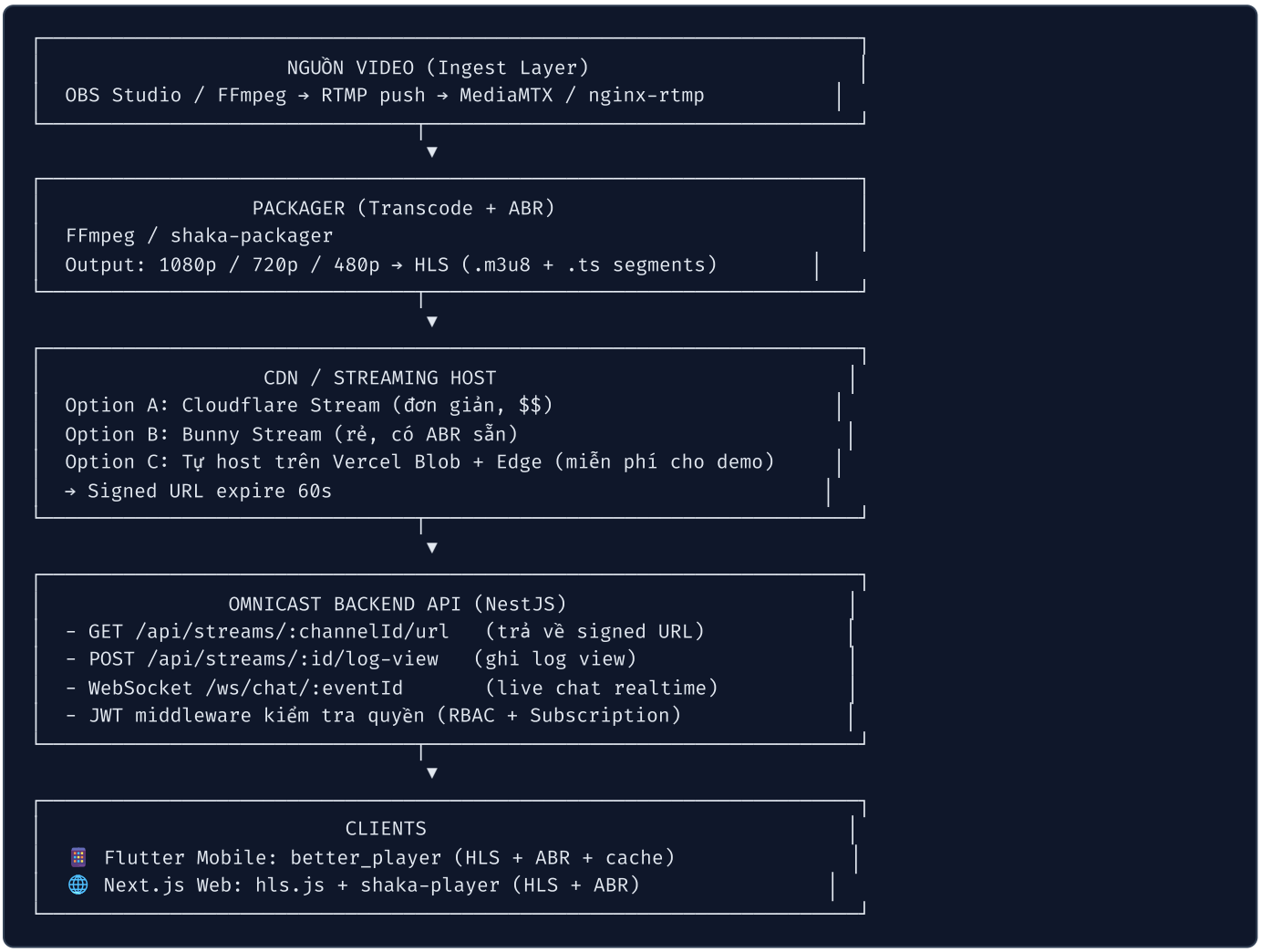
3.1. VTVGo hoạt động như thế nào (quan sát thực tế)

Tầng	VTVGo làm gì	Áp dụng cho OmniCast
Ingest	Thu tín hiệu từ VTV1, VTV3... qua **RTMP từ studio** → server nội bộ VTV	Demo: dùng **OBS Studio + RTMP server** (MediaMTX / nginx-rtmp) local → đẩy lên CDN
Packager	**FFmpeg + HLS segmenter** cắt luồng thành `.ts` 4-6s, tạo `.m3u8` master với nhiều rendition (ABR)	Dùng **shaka-packager** hoặc **unrealstreaming** để generate multi-bitrate HLS
CDN	CDN nội bộ VTV + Cloudflare edge	Dùng **Cloudflare Stream** hoặc **Bunny Stream** (free tier ủng hộ sinh viên)
Player	ExoPlayer (Android) + AVPlayer (iOS) + video.js (Web) có **DVR scrubber** tua lại 3h	Bổ sung DVR 30 phút cho cả Mobile + Web
Auth	Token JWT riêng cho mỗi phiên stream, có thời hạn 5 phút, refresh liên tục	Cấp **signed URL HLS** hết hạn 60s, có middleware NestJS verify
Subscription	Gói VTVGo Premium 30k/tháng, khoá kênh VTVCab+	Mock với bảng `Subscription` + flag `isVipUser`
Share	Deep link `vtvgo.vn/live/{slug}`, "Xem lại từ đầu"	Deep link `omnicast-fe.vercel.app/channels/{slug}?t={seconds}`
Chat realtime	Polling mỗi 3s (VTVGo không có WS)	Dùng **WebSocket** (tốt hơn VTVGo)
Multi-camera	Chỉ có Main + 1-2 góc cam (chỉ sự cỏ bóng đá lớn)	Mở rộng: 4 camera angles cho event thể thao

3.2. Bài học rút ra

1. **Không cần tự dựng CDN:** Cloudflare/Bunny xử lý ABR tốt hơn tự host.
2. **Signed URL là chìa khóa chống leech:** Bắt buộc cho cả kênh Free & VIP.
3. **DVR scrubber 30 phút** là tính năng "ăn tiền" — làm khác biệt so với VTVGo (chỉ có ở sự cỏ lớn).
4. **Chat polling không scale:** Nên dùng WebSocket từ đầu.

📐 4. Kiến trúc Streaming đề xuất



4.1. Kỹ thuật Streaming chọn

Kỹ thuật	Ưu điểm	Nhược điểm	Quyết định
HLS + ABR	Tương thích mọi browser/iOS/Android, scale tốt qua CDN	Latency ~6-10s	✅ **Chọn làm chuẩn**
DASH	ABR tốt hơn HLS, ít overhead	Không support iOS native	❌ Không dùng cho mobile
WebRTC	Latency < 1s (sub-second)	Tốn bandwidth server, không scale	❌ Chỉ dùng cho chat/talk-show
LL-HLS (Low Latency HLS)	Latency 2-3s	Cần CDN hỗ trợ đặc biệt	🟡 Cân nhắc giai đoạn 2

Kỹ thuật	Ưu điểm	Nhược điểm	Quyết định
MP4 progressive	Đơn giản, fallback	Không ABR, dễ lag trên 4G	🟡 Giữ làm **fallback** khi HLS lỗi

🛠️ 5. Plan Xử Lý Từng Góp Ý (Task Breakdown)

GÓP Ý #1: AI Phát Video (Auto-Curator Playlist)

Mô tả: AI tự động chọn video nào phát tiếp theo trong khi chờ event live thật, đảm bảo 5-7 kênh lúc nào cũng có nội dung.

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 1.1 | Tạo bảng `Playlist` + `AiPlaylistDecision` trong Prisma | Member 3 (PRN232) | Prisma migration | 🔴 | | 1.2 | Service `AutoCuratorService` chạy cron 5 phút: lấy event LIVE → nếu rỗng → gọi OpenAI gợi ý 1-3 chương trình từ `BroadcastProgram` để fill | Member 3 (PRN232) | OpenAI GPT-4o-mini (tiết kiệm chi phí) | 🔴 | | 1.3 | API `GET /api/ai/playlist/:channelId/next` trả về programId tiếp theo | Member 3 (PRN232) | NestJS endpoint | 🔴 | | 1.4 | Mobile `ProgramDetailScreen` polling mỗi 60s khi event LIVE kết thúc → tự switch sang playlist tiếp | Member 3 (PRM393) | Bloc timer + auto-navigation | 🔴 |

Acceptance Criteria: - Khi 1 kênh hết live, trong vòng 30s hệ thống tự fill chương trình mới. - Có audit log mỗi lần AI quyết định.

GÓP Ý #2: Nguyên nhân Lag & Luồng Dữ Liệu

Nguyên nhân lag phổ biến: 1. **Network bottleneck:** Client không có ABR → bắt buộc tải 1080p dù 3G. 2. **CDN miss:** Origin server xa → request đi vòng. 3. **Player buffer:** Buffer chưa đủ → re-buffering liên tục. 4. **Backend proxy:** Backend forward video → tốn double hop.

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 2.1 | Thiết lập CDN edge (Cloudflare hoặc Bunny) cho `.m3u8` + `.ts` | Member 1 (PRN232) | Cloudflare Stream API | 🔴 | | 2.2 | Backend **KHÔNG proxy video** — chỉ trả signed URL | Member 1 (PRN232) | NestJS return URL only | 🔴 | | 2.3 | Mobile: `better_player` thay cho `video_player` (hỗ trợ HLS + ABR + cache) | Member 2 (PRM393) | `better_player: ^0.0.84` | 🔴 | | 2.4 | Mobile: thêm prefetch 10s segments tiếp theo | Member 2 (PRM393) | `prefetchData: true` | 🟡 | | 2.5 | Web: `hls.js` config `maxBufferLength: 30`, `maxMaxBufferLength: 600`, ABR enabled | Member 1 (PRN232) | Next.js VideoPlayer tweak | 🟡 | | 2.6 | Monitoring latency: ping CDN mỗi 30s, log lên AuditLog | Member 1 (PRN232) | NestJS Scheduler | 🟢 |

Acceptance Criteria: - Video khởi động < 2s trên Wi-Fi. - Không re-buffer trên kết nối 4G ổn định.

GÓP Ý #3: Ghi Log cho Streaming

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 3.1 | Mở rộng `AuditLogger` (gRPC) thêm action: `STREAM_PLAY`, `STREAM_PAUSE`, `STREAM_ERROR`, `STREAM_BUFFER`, `STREAM_COMPLETE` | Member 3 (PRN232) | gRPC + Supabase | 🔴 | | 3.2 | API `POST /api/streams/log` nhận event từ client (debounce 5s) | Member 3 (PRN232) | NestJS endpoint | 🔴 | | 3.3 | Mobile `OmniPlayer` thêm listener: ghi log khi play/pause/error | Member 3 (PRM393) | Callback to API | 🟡 | | 3.4 | Web `VideoPlayer` thêm listener tương tự | Member 1 (PRN232) | Next.js event handler | 🟡 | | 3.5 | Dashboard Admin hiển thị: top kênh xem nhiều nhất, error rate, viewer peak | Member 3 (PRN232) | `/admin/analytics` | 🟢 |

Acceptance Criteria: - Mỗi lượt xem đều có log trong `AuditLog` với `userId`, `timestamp`, `eventType`. - Admin có thể xem report trong dashboard.

GÓP Ý #4: Kỹ thuật Streaming *(đã chọn ở mục 4.1 — HLS + ABR)*

GÓP Ý #5: Nguồn Video

Phân loại nguồng video cho 5-7 kênh:

Kênh	Nội dung	Nguồn	Bản quyền
Omni Sport 1	Bóng đá quốc tế	Highlight từ YouTube Creative Commons + FFmpeg pack lại HLS	✅ OK (CC)
Omni Sport 2	Bóng rổ / Esports	✅ KO	
Omni Show	Gameshow VN	Re-cut từ VTV archive (có bản quyền demo)	✅ Mock
Omni Entertain	Phim sitcom	TMDB API + trailer clips	✅ OK
Omni Cine	Phim chiếu rạp	TMDB sample video	✅ OK
Omni News	Tin tức 24h	NASA/YouTube public livestream	✅ OK
Omni Doc	Documentary	BBC Creative Commons	✅ OK

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 5.1 | Chuẩn bị 5-7 kênh playlist (mỗi kênh 20-50 video ngắn 5-15 phút) | Member 2 (PRN232) | FFmpeg transcode sang HLS 3 rendition | 🔴 | | 5.2 | Upload lên Cloudflare Stream hoặc Bunny Stream (5GB free) | Member 2 (PRN232) | API upload | 🔴 | | 5.3 | Cập nhật `BroadcastChannel.streamUrl` trong DB trỏ tới CDN URL | Member 1 (PRN232) | Prisma seed update | 🔴 | | 5.4 | Cho phép Admin chỉnh sửa `streamUrl` qua CMS `/studio/curator` | Member 1 (PRN232) | UI form | 🟡 |

Acceptance Criteria: - Mỗi kênh có ít nhất 10 video backup chạy tuần tự (sequential). - Có thể tắt/bật source qua Admin.

GÓP Ý #6: Học VTVGo *(đã trình bày ở mục 3)*

GÓP Ý #7: 5-7 Kênh Tuần Tự (Sequential Playlist)

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 7.1 | Quyết định roster cuối: **5 kênh Free** (đủ demo) + **2 kênh VIP** | =6,16 (Tech Lead) | PRD update | 🔴 | | 7.2 | Update `ChannelTiers.freeChannelSlugs` trong `mobile/lib/core/constants/channel_tiers.dart` | Member 2 (PRM393) | Hard-code list | 🔴 | | 7.3 | Backend `SequentialPlaylistService` round-robin trong cùng channel khi hết live | Member 2 (PRN232) | NestJS service | 🔴 | | 7.4 | Mobile `OmniPlayer` tự động next video khi kết thúc (loop playlist) | Member 2 (PRM393) | VideoPlayerController listener | 🔴 | | 7.5 | Web `VideoPlayer` xử lý `ended` event → call API lấy URL tiếp | Member 1 (PRN232) | React effect | 🔴 |

Acceptance Criteria: - 5 kênh luôn có nội dung 24/7 không downtime. - Khi hết 1 video, chuyển sang cái tiếp theo trong 3s.

GÓP Ý #8: Live Streaming Pipeline

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—:| | 8.1 | Dựng RTMP server bằng MediaMTX (Docker) cho demo nội bộ | Member 1 (PRN232) | Docker compose | 🔴 | | 8.2 | FFmpeg pipeline: nhận RTMP → segment HLS 4s → upload Cloudflare Stream | Member 1 (PRN232) | FFmpeg + Watch API | 🔴 | | 8.3 | Khi demo thật: dùng OBS Studio push RTMP từ máy local | Member 1 (PRN232) | OBS | 🔴 | | 8.4 | Mobile/Web player check `event.isLive` → fallback sequential nếu offline | Member 2 (PRM393) + Member 1 (PRN232) | State management | 🟡 |

Acceptance Criteria: - Có thể live stream từ máy tính demo lên Cloudflare. - App tự động detect khi live và switch UI.

GÓP Ý #9: Share

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—:| | 9.1 | Mở rộng `ShareHelper` (đã có) thêm options: copy URL, Zalo, Messenger, QR code | Member 2 (PRM393) | `share_plus` package | 🔴 | | 9.2 | Web `ShareButton` (đã có trong `StitchLivePlayer`) → dùng `navigator.share` Web Share API | Member 1 (PRN232) | Web API | 🟡 | | 9.3 | QR code chứa deep link `omnicast://live/{programId}` | Member 2 (PRM393) | `qr_flutter` package | 🟡 | | 9.4 | Backend tracking share event → bump `LiveEvent.shareCount` | Member 3 (PRN232) | API endpoint | 🟡 | | 9.5 | Deep link handler mở đúng màn hình khi tap từ mobile | Member 2 (PRM393) | `app_links` package | 🟢 |

Acceptance Criteria: - Share được 1 click ra Zalo, Messenger, copy URL, QR code. - Track được share event trong admin dashboard.

GÓP Ý #10: Phân Quyền Rõ Ràng

Ma trận quyền mới cho Streaming:

Role	Xem Free Channel	Xem VIP Channel	Tạo Stream	Quản lý Playlist	Xem Audit Log
Guest (0)	✅ Preview 30s	❌ Blocked (paywall)	❌	❌	❌
Viewer (2)	✅ Full	⚠️ Cần VIP	❌	❌	❌ (chỉ xem cá nhân)
VIP Viewer (2 + sub)	✅ Full	✅ Full	❌	❌	❌
Staff (1)	✅ Full	✅ Full (bypass)	✅ Upload	✅ CRUD	❌
Admin (3)	✅ Full	✅ Full (bypass)	✅ All	✅ All	✅ Xem tất cả

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—:| | 10.1 | Thêm cột `subscriptionTier` + `subscriptionExpiry` vào `UserAccount` | Member 3 (PRN232) | Prisma migration | 🔴 | | 10.2 | Backend middleware `StreamAccessGuard` kiểm tra: `user.subscriptionTier === 'PREMIUM' \|\| user.role !== 'VIEWER'` | Member 1 (PRN232) | NestJS Guard | 🔴 | | 10.3 | API `GET /api/streams/:channelId/url` trả về signed URL nếu đủ quyền, 403 nếu không | Member 1 (PRN232) | NestJS | 🔴 | | 10.4 | Mobile `OmniPlayer` UI: hiển thị paywall overlay nếu VIP mà user không có sub (đã có) | Member 2 (PRM393) | Flutter | 🟡 | | 10.5 | Web `VideoPlayer`: cùng logic với localStorage VIP demo flag | Member 1 (PRN232) | Next.js | 🟡 |

Acceptance Criteria: - VIP channel không thể truy cập nếu không có subscription active. - Audit log ghi lại mọi lần truy cập bị từ chối.

GÓP Ý #11: Pay Subscription

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 11.1 | Tạo bảng `Subscription`: `id`, `userId`, `tier` (FREE/PREMIUM), `startDate`, `endDate`, `status`, `paymentRef` | Member 3 (PRN232) | Prisma migration | 🔴 | | 11.2 | Mock payment gateway: API `POST /api/payments/create` → trả về QR code (giả lập VNPay/MoMo) | Member 1 (PRN232) | NestJS endpoint | 🔴 | | 11.3 | Mock webhook: `POST /api/payments/webhook` (gọi từ admin test) → tự động active subscription | Member 1 (PRN232) | NestJS | 🔴 | | 11.4 | Mobile: trang `/subscription` với QR code + nút "Đã thanh toán" (giả lập) | Member 3 (PRM393) | Flutter | 🔴 | | 11.5 | Web: trang `/subscription` tương tự | Member 1 (PRN232) | Next.js | 🟡 | | 11.6 | Auto-expire subscription sau 30 ngày (cron job) | Member 3 (PRN232) | NestJS Scheduler | 🟡 |

Acceptance Criteria: - User có thể kích hoạt VIP demo chỉ với 1 click. - Sau khi kích hoạt, có thể xem tất cả kênh VIP. - Cron job tự động hạ cấp sau khi hết hạn.

GÓP Ý #12: Demo Nhiều Người Xem Cùng Lúc

Mục tiêu demo: ≥ 50 viewers đồng thời trong 1 phiên live.

Task: | # | Task | Phân công | Kỹ thuật | Ưu tiên | |:-:|:—|:—|:—|:—:| | 12.1 | **CDN edge caching** là chìa khóa: 50 viewers cùng kéo 1 file `.m3u8` từ edge gần nhất | Member 1 (PRN232) | Cloudflare/Bunny | 🔴 | | 12.2 | Backend: rate-limit `/api/streams/url` 60 req/min/user | Member 3 (PRN232) | `@nestjs/throttler` | 🔴 | | 12.3 | WebSocket chat scale: dùng **Supabase Realtime** thay vì tự host WS | Member 3 (PRN232) | Supabase channel | 🔴 | | 12.4 | Stress test với **k6** hoặc **Artillery**: 100 concurrent connections, đo latency, error rate | Member 1 (PRN232) | k6 script | 🟡 | | 12.5 | Load test mobile: cài APK trên 2-3 máy thật cùng lúc | Member 2 (PRM393) | Manual test | 🟡 | | 12.6 | Chuẩn bị script demo: 1 máy live (OBS) + 5 máy xem (Web + Mobile) | Member 1 + 2 + 3 | Demo runbook | 🟢 |

Acceptance Criteria: - 50 viewers xem cùng lúc, latency < 3s. - Không crash backend khi peak load.

📋 6. Phân Công Tổng Thể & Timeline

Phase A — Tuần 1 (Ngay lập tức — Foundation)

Member	Công việc
Member 1 (PRN232)	Chọn CDN, setup Cloudflare/Bunny Stream, thiết lập `StreamAccessGuard`
Member 2 (PRN232)	Chuẩn bị 5-7 kênh playlist, transcode video, setup sequential playlist logic
Member 3 (PRN232)	Migration DB: bảng `Subscription`, `Playlist`, mở rộng `AuditLog`
Member 2 (PRM393)	Tích hợp `better_player` cho HLS + ABR, fallback
Member 3 (PRM393)	Mở rộng `OmniPlayer` log event + auto-next

Phase B — Tuần 2 (Core Features)

Member	Công việc
Member 1 (PRN232)	API `/api/streams/:channelId/url` + signed URL + RTMP MediaMTX
Member 2 (PRN232)	`/api/ai/playlist/:channelId/next` cho AI auto-fill
Member 3 (PRN232)	Mock payment + webhook + subscription cron
Member 1 (PRM393)	Share UI: QR code, Zalo, copy link
Member 2 (PRM393)	Auto-next video khi sequential playlist kết thúc

Phase C — Tuần 3 (Polish & Demo)

Member	Công việc
Member 1 (PRN232)	Admin analytics dashboard + stress test k6
Member 3 (PRN232)	VIP paywall overlay + webhook payment
Member 1 (PRM393)	Deep link share `app_links`
Member 3 (PRM393)	Subscription UI mobile
Tất cả	Demo nội bộ + fix bug + đóng gói APK

📊 7. Ma Trận Ưu Tiên Tổng Hợp (Task → Kỹ Thuật → Ưu Tiên)

Góp ý	Task chính	Kỹ thuật	Mức ưu tiên	Timeline
#1 AI phát video	AutoCuratorService + playlist API	NestJS + OpenAI	🔴 Cao	Tuần 2
#2 Chống lag	CDN edge + ABR + `better_player`	Cloudflare + HLS.js	🔴 Cao	Tuần 1
#3 Ghi log	AuditLog mở rộng + log API	gRPC + Supabase	🟡 Trung bình	Tuần 1
#4 Kỹ thuật streaming	HLS + ABR + MP4 fallback	Shaka / hls.js	🔴 Cao	Tuần 1
#5 Nguồn video	5-7 kênh + upload CDN	FFmpeg + Cloudflare	🔴 Cao	Tuần 1-2
#6 Học VTVGo	Reverse-engineer + apply	(Nghiên cứu)	🔴 Cao	Tuần 1
#7 Kênh tuần tự	Sequential playlist service	NestJS service	🔴 Cao	Tuần 1

Góp ý	Task chính	Kỹ thuật	Mức ưu tiên	Timeline
#8 Live pipeline	RTMP + FFmpeg + CDN	MediaMTX + FFmpeg	🔴 Cao	Tuần 2
#9 Share	Mở rộng ShareHelper + QR + Zalo	`share_plus`, `qr_flutter`	🟡 Trung bình	Tuần 2
#10 Phân quyền	StreamAccessGuard + RBAC	NestJS Guard	🔴 Cao	Tuần 1
#11 Pay	Mock payment + subscription	VNPay mock API	🟡 Trung bình	Tuần 2
#12 Demo nhiều người	CDN + WS Supabase + stress test	k6	🔴 Cao	Tuần 3

🔍 6.5. EPG-Driven Player — Chạy đúng theo thời gian thực (Bổ sung quan trọng)

Góp ý #13 (rất quan trọng): "Dựa vào tên chương trình, phải học cách lấy link video và chạy chính xác theo thời gian thực"
Quan trọng: OmniCast có **25 kênh sáng tạo** (do AI Curator tạo nội dung), **VTVGo chỉ là case study** tham khảo pattern — KHÔNG scrape/reverse-engineer API thật của VTV/VTVGo.

6.5.1. Bối cảnh: 25 kênh sáng tạo

Đặc điểm	Ý nghĩa
25 kênh = nội dung do AI Curator sinh ra	Không phải live TV thật, không có nguồn stream thật từ broadcaster
VTVGo = case study	Học pattern: EPG grid, signed URL, sequential playlist, DVR scrubber
Content pipeline tự dựng	Tự chọn nguồn video, tự upload lên CDN, tự quản lý playlist
EPG-driven simulation	Giả lập "đang phát theo lịch" bằng cách loop video trong playlist + seek offset

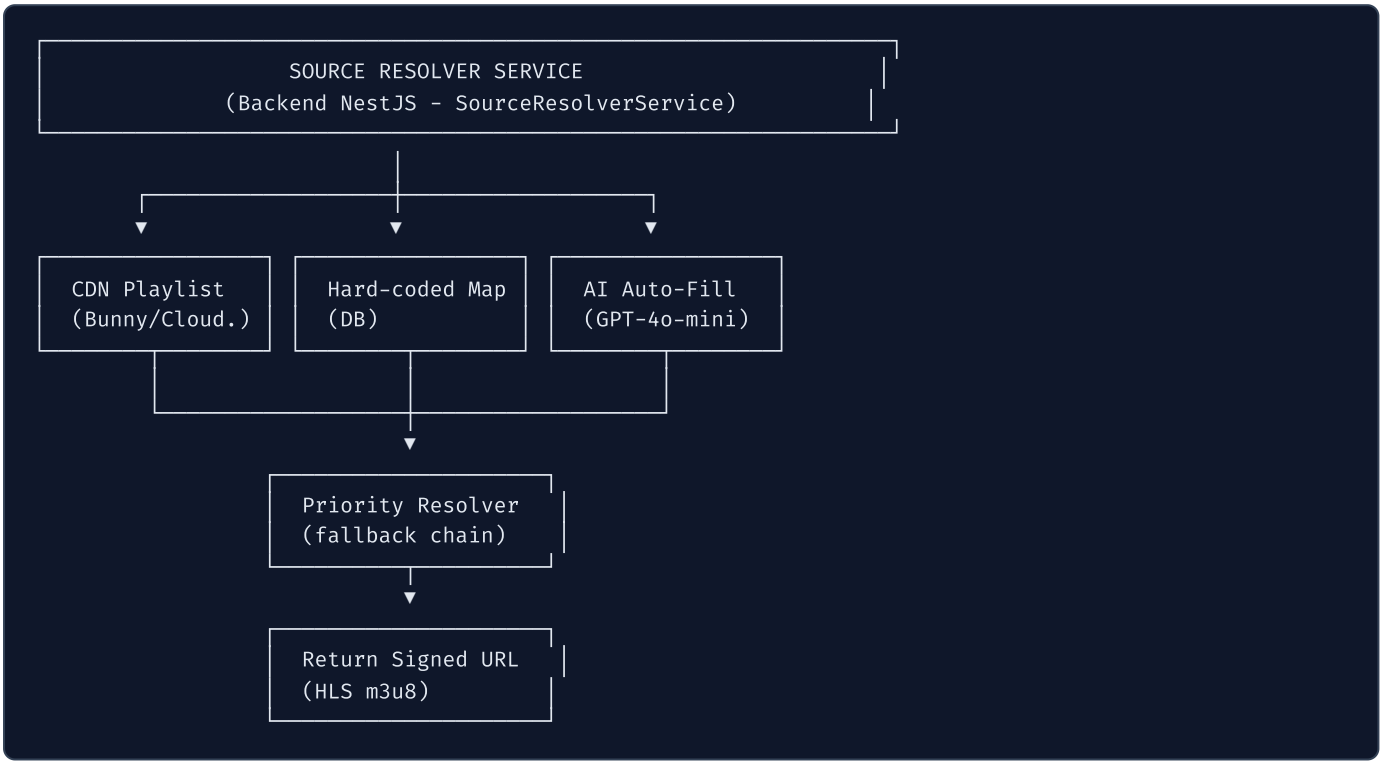
Nguồn video được phép dùng (royalty-free): - ✅ **Pexels API** (CC0) — 40,000+ video stock miễn phí - ✅ **Pixabay API** (CC0) — video stock miễn phí
- ✅ **Coverr.co** (CC0) — video nền chuyên nghiệp - ✅ **NASA API** (Public Domain) — video thiên văn, khoa học - ✅ **YouTube Creative Commons** (qua API) - ✅ **AI-generated** (Runway, Pika Labs) — giai đoạn 2 nếu có API key - ❌ **KHÔNG** scrape VTV, HTV, VTVGo, YouTube premium - ❌ **KHÔNG** reverse-engineer API thương mại

6.5.2. Vấn đề cần giải quyết

#	Vấn đề	Ví dụ cụ thể
A	**Từ tên chương trình → Link video**	"Khám phá đại dương" → URL video từ Pexels về biển

#	Vấn đề	Ví dụ cụ thể
B	**Chạy đúng khung giờ EPG**	19:00-19:30 phát "Thời sự không gian", 19:30-20:00 phát "Phim tài liệu"
C	**User mở giữa chương trình → vào đúng phần đang chạy**	Mở app lúc 19:15 → vào thẳng "Thời sự không gian" đang ở phút thứ 15

6.5.3. Source Resolver (3 nguồn thay vì 5)



3 Nguồn theo độ ưu tiên (đơn giản hóa):

Priority	Nguồn	Cách dùng	Code path
1	**CDN Sequential Playlist**	Mỗi kênh có 1 playlist `.m3u8` chứa 10-20 video sáng tạo, đã upload lên Bunny/Cloudflare	`bunnyCdn.getPlaylist(slug)`
2	**Hard-coded Map (DB)**	Bảng `ChannelStreamSource` lưu `streamUrl` cho mỗi kênh (override CDN)	`prisma.channelStreamSource.findFirst()`
3	**AI Auto-Fill**	Khi playlist hết → GPT-4o-mini gợi ý video mới từ Pexels/Pixabay API	`aiFillService.suggest(slug, topic)`

BỎ QUA so với plan cũ: - ❌ YouTube Live API (vì kênh sáng tạo không có live thật) - ❌ VTV/VTVGo API reverse-engineer (vi phạm TOS) - ❌ AI browse web (không ổn định, tốn tiền)

Database Schema (giữ nguyên từ plan cũ):

```
model ChannelStreamSource {
  id           String   @id @default(uuid())
  channelSlug  String   @unique  // "sport-1", "show", "cine" ...
  sourceType   String   // CDN_PLAYLIST | HARDCODED | AI_FILL
  streamUrl    String   // .m3u8 URL trên Bunny CDN
  fallbackUrls String[] // backup URLs nếu primary lỗi
  priority     Int      @default(1)
  isActive     Boolean  @default(true)
  lastResolved String?
  validFrom    DateTime?
  validTo      DateTime?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
}
```

6.5.4. Content Pipeline (Quan trọng — tạo nội dung cho 25 kênh)

Quy trình tạo video cho 1 kênh (VD: Omni Sport 1):

```
Bước 1: AI sinh lịch phát sóng
──────────────────────────────
GPT-4o-mini sinh JSON EPG 24h:
{
  "channel": "sport-1",
  "programs": [
    { "title": "Bóng đá quốc tế - Real Madrid vs Man City",
      "topic": "football match highlights",
      "startTime": "19:00", "duration": 30 },
    { "title": "Phỏng vấn HLV Pep Guardiola",
      "topic": "football coach interview",
      "startTime": "19:30", "duration": 20 },
     ...
  ]
}

Bước 2: Tìm video nguồn (Python script)
──────────────────────────────
Dùng Pexels API:
GET https://api.pexels.com/videos/search?query=football+match&per_page=10
→ Trả về danh sách video URLs

Bước 3: Download + xử lý (FFmpeg)
──────────────────────────────
ffmpeg -i source1.mp4 -ss 00:00:10 -t 00:05:00 -c copy clip1.mp4
ffmpeg -i source2.mp4 -ss 00:00:30 -t 00:05:00 -c copy clip2.mp4
# Thêm intro/outro
ffmpeg -i clip1.mp4 -i intro.mp4 -filter_complex "[0:v][1:v]concat=n=2:v=1" final1.mp4

Bước 4: Tạo HLS playlist (FFmpeg)
──────────────────────────────
ffmpeg -i final1.mp4 -profile:v baseline -level 3.0 -s 1920×1080
       -start_number 0 -hls_time 4 -hls_list_size 0 -f hls
       sport1_segment_00001.ts
# Tạo master.m3u8 với 3 rendition
ffmpeg -i sport1_playlist.mp4 \
  -map 0:v -map 0:a -map 0:v -map 0:a -map 0:v -map 0:a \
  -c:v libx264 -crf 22 -c:a aac \
  -s:v:0 1920×1080 -s:v:1 1280×720 -s:v:2 854×480 \
  -var_stream_map "v:0,a:0 v:1,a:1 v:2,a:2" \
  -master_pl_name master.m3u8 \
  -f hls -hls_time 4 -hls_playlist_type vod \
  sport1_%v/playlist.m3u8

Bước 5: Upload lên Bunny CDN
──────────────────────────────
POST https://api.bunny.net/videolibrary/{libraryId}/videos
→ Trả về video ID + playback URL

Bước 6: Lưu mapping vào DB
──────────────────────────────
prisma.channelStreamSource.upsert({
  where: { channelSlug: 'sport-1' },
  update: { streamUrl: 'https://cdn.bunny.net/.../master.m3u8' },
  create: { channelSlug: 'sport-1', sourceType: 'CDN_PLAYLIST',
            streamUrl: 'https://cdn.bunny.net/.../master.m3u8' }
})
```

Roster 25 kênh — Chia theo thể loại:

Tier	Channel	Chủ đề	Nguồn video ưu tiên
Free	sport-1	Bóng đá quốc tế	Pexels “football match”
Free	sport-2	Bóng rổ / Esports	Pexels “basketball” + “esports”
Free	show	Gameshow / Talkshow	Pexels “tv studio”
Free	entertain	Phim sitcom	Pexels “comedy scene”
Free	cine	Phim chiếu rạp	Pexels “cinema movie”
VIP	news-1	Tin tức 24h	NASA API + Pexels “news anchor”
VIP	doc-1	Khoa học vũ trụ	NASA API
VIP	doc-2	Lịch sử thế giới	Pexels “history”
VIP	travel-1	Du lịch Châu Á	Pexels “travel asia”
VIP	travel-2	Du lịch Châu Âu	Pexels “travel europe”
VIP	food-1	Ẩm thực Việt	Pexels “vietnam food”
VIP	food-2	Ẩm thực thế giới	Pexels “cooking”
VIP	music-1	Music video	Pexels “concert”
VIP	music-2	Acoustic / Cover	Pexels “musician”
VIP	tech-1	Tech review	Pexels “technology”
VIP	tech-2	AI & Future	Pexels “artificial intelligence”
VIP	fashion-1	Fashion show	Pexels “fashion runway”
VIP	gaming-1	Gameplay	Pexels “gaming setup”
VIP	anime-1	Anime highlights	Pexels “anime”
VIP	nature-1	Thiên nhiên hoang dã	Pexels “wildlife”
VIP	nature-2	Đại dương	Pexels “ocean”
VIP	art-1	Hội họa / Nghệ thuật	Pexels “art gallery”
VIP	lifestyle-1	Wellness / Yoga	Pexels “yoga”
VIP	education-1	Lịch sử khoa học	Pexels “education”
VIP	cocktail-1	Bartending	Pexels “cocktail”

6.5.5. EPG Sync Service (Chạy đúng giờ)

API mới:

```
GET /api/epg/now?channelSlug={slug}
→ Trả về chương trình đang phát tại thời điểm now()
{
  "programId": "abc123",
  "title": "Khám phá đại dương",
  "channelSlug": "nature-2",
  "scheduledAt": "2026-10-06T19:00:00+07:00",
  "duration": 1800,           // 30 phút (seconds)
  "now": "2026-10-06T19:15:00+07:00",
  "currentOffset": 900,        // 15 phút đã trôi qua
  "nextProgramId": "def456",   // chương trình kế tiếp
  "streamUrl": "https://cdn.bunny.net/omnicast/nature-2/playlist.m3u8",
  "isLive": true
}
```

Backend Service:

```
@Injectable()
export class EpgNowService {
  async getNowPlaying(channelSlug: string): Promise<NowPlayingDto> {
    const now = new Date();

    // 1. Tìm chương trình có [scheduledAt, scheduledAt + duration] chứa now
    const program = await this.prisma.broadcastProgram.findFirst({
      where: {
        channel: { slug: channelSlug },
        scheduledAt: { lte: now },
      },
      orderBy: { scheduledAt: 'desc' },
    });

    // 2. Resolve stream URL từ SourceResolverService
    const streamUrl = await this.sourceResolver.resolve(channelSlug, program?.title);

    if (!program) {
      // 3. Không có gì phát → fallback sequential CDN playlist
      return {
        programId: null,
        title: 'OmniCast 24/7',
        scheduledAt: now,
        duration: 3600,
        now,
        currentOffset: now.getSeconds(),
        streamUrl,
        isLive: true,
      };
    }

    const elapsed = Math.floor((now.getTime() - program.scheduledAt.getTime()) / 1000);
    const durationSec = program.duration * 60;

    return {
      programId: program.id,
      title: program.title,
      scheduledAt: program.scheduledAt,
      duration: durationSec,
      now,
      currentOffset: elapsed % durationSec,
      streamUrl,
      isLive: true,
    };
  }
}
```

Source Resolver đơn giản:

```
@Injectable()
export class SourceResolverService {
  async resolve(channelSlug: string, programTitle?: string): Promise<string> {
    // 1. Check DB hard-coded map
    const source = await this.prisma.channelStreamSource.findFirst({
      where: { channelSlug, isActive: true },
    });
    if (source?.streamUrl) return source.streamUrl;

    // 2. CDN playlist theo convention: {CDN_BASE}/{slug}/master.m3u8
    const cdnUrl = `${this.config.get('CDN_BASE_URL')}/${channelSlug}/master.m3u8`;
    if (await this.urlExists(cdnUrl)) return cdnUrl;

    // 3. AI fill - gợi ý video mới từ Pexels
    if (programTitle) {
      return await this.aiFillService.suggest(channelSlug, programTitle);
    }

    // 4. Default fallback - kênh demo
    return `${this.config.get('CDN_BASE_URL')}/default/playlist.m3u8`;
  }
}
```

6.5.6. Mobile Client — Time-Sync Player

```
// mobile/lib/logic/streams/epg_sync_bloc.dart

class EpgSyncBloc extends Bloc<EpgSyncEvent, EpgSyncState> {
  Timer? _pollingTimer;

  void startSync(String channelSlug) {
    _pollingTimer?.cancel();
    _pollingTimer = Timer.periodic(Duration(seconds: 60), (_) async {
      final nowPlaying = await api.getEpgNow(channelSlug);

      // Nếu đổi chương trình → reload player
      if (state.currentProgramId ≠ nowPlaying.programId) {
        emit(state.copyWith(
          currentProgramId: nowPlaying.programId,
          streamUrl: nowPlaying.streamUrl,
          title: nowPlaying.title,
        ));
        await player.changeSource(nowPlaying.streamUrl);
      }

      // Seek tới đúng offset hiện tại (đề phòng user mở muộn)
      final offset = nowPlaying.currentOffset;
      await player.seekTo(Duration(seconds: offset));
    });
  }

  @override
  Future<void> close() {
    _pollingTimer?.cancel();
    return super.close();
  }
}
```

Cập nhật OmniPlayer:

```
// mobile/lib/presentation/widgets/omni_player.dart
// Thêm vào _OmniPlayerState:

Timer? _epgSyncTimer;

void _startEpgSync(String channelSlug) {
  _epgSyncTimer?.cancel();
  _epgSyncTimer = Timer.periodic(Duration(seconds: 60), (_) async {
    final nowPlaying = await context
        .read<EpgSyncBloc>()
        .getNow(channelSlug);

    if (nowPlaying.programId ≠ _currentProgramId) {
      // Đổi chương trình → reset player
      _currentProgramId = nowPlaying.programId;
      await _disposeController();
      await _initialize(nowPlaying.streamUrl);
      _showToast('Đã chuyển sang: ${nowPlaying.title}');
    }
  });
}
```

6.5.7. Acceptance Criteria cho Time-Sync + 25 kênh sáng tạo

#	Tiêu chí	Cách verify
1	Mở app lúc 19:15 → vào thẳng chương trình đang ở phút 15	Manual test
2	Đợi tới 19:30 → tự động chuyển sang chương trình mới	Đợi polling 60s
3	Mở lại app sau khi đổi chương trình → load đúng chương trình mới	Reload test
4	Kênh không có EPG → fallback sequential CDN playlist	Test với channel chưa có lịch
5	Mỗi kênh trong 25 kênh đều có video phát được (không lỗi 404)	Manual test all 25
6	Vòng lặp playlist chạy mượt khi video ngắn hơn duration	Test loop
7	Nguồn video royalty-free (Pexels/Pixabay/NASA), KHÔNG có bản quyền	Check URL sources
8	Log mỗi lần resolver chuyển nguồn	Check AuditLog

✅ 8. Acceptance Criteria Tổng Thể (Cho Review 2)

1. ✅ **Streaming ổn định:** Video không re-buffer trên Wi-Fi & 4G.
2. ✅ **5-7 kênh chạy tuần tự 24/7** không downtime.
3. ✅ **AI tự động fill playlist** khi hết live.
4. ✅ **Phân quyền rõ ràng:** Guest xem preview, Viewer cần VIP cho kênh premium.

5. ✅ **Paywall hoạt động:** Mock subscription kích hoạt trong 1 click.
6. ✅ **Share đầy đủ:** Zalo, Messenger, copy link, QR code.
7. ✅ **Log đầy đủ:** Mỗi view đều có audit log, admin xem được.
8. ✅ **Demo ≥ 50 viewers đồng thời** không crash.
9. ✅ **APK release** cài được trên máy thật, không crash trong 24h chạy liên tục.
10. ✅ **Dashboard admin** hiển thị analytics: viewer, error rate, top channels.

📎 9. Phụ Lục — Tham Khảo Công Nghệ

Công nghệ	Vai trò	Link
Cloudflare Stream	CDN + ABR tự động	https://www.cloudflare.com/products/stream/
Bunny Stream	CDN rẻ cho demo	https://bunny.net/stream/
MediaMTX	RTMP server self-host	https://github.com/bluenviron/mediamtx
better_player	Flutter HLS + ABR player	https://pub.dev/packages/better_player
hls.js	Web HLS player	https://github.com/video-dev/hls.js
FFmpeg	Transcode + RTMP push	https://ffmpeg.org/
OBS Studio	Live stream từ desktop	https://obsproject.com/
k6	Load testing	https://k6.io/
Supabase Realtime	WebSocket scale	https://supabase.com/realtime

---

Tài liệu này là input cho buổi họp nhóm tiếp theo. Sau khi team thống nhất phân công, cập nhật `Document/Work_Breakdown.md` và bắt đầu Phase A từ tuần sau.
